

Horizon–Goldbach TS21–TS29 Bundle

Short-Interval Scale Correction, OTSA Propagation, and Rational Constant Registers

A continuation bundle after TS16–TS19, with all TS21–TS29 Lean sources

Serge Durand
Horizon / Goldbach Lean Programme

Baseline: Lean 4.15.0 / Mathlib v4.15.0
Generated bundle date: 2026-05-27

Abstract

This bundle continues the formal documentation sequence after TS16–TS19. It covers TS21–TS29 of the Horizon–Goldbach Lean programme, concentrating on the scale correction of the short-interval second-moment interface, the padded closed-form Brun–Titchmarsh scale bridge, the OTSA scale propagation layer, and the rational constant-register machinery. The document is arXiv-style and fail-closed: it does not claim an unconditional proof of Goldbach. It records the mechanically checked architecture and the remaining analytic obligations. All Lean sources for TS21–TS29 are included verbatim in the appendices.

Contents

1	Epistemic status and non-claims	3
2	Position after TS16–TS19	3
3	TS21–TS29 overview	3
4	The scale correction	4
5	Brun–Titchmarsh and the padded closed form	4
6	OTSA propagation and rational certificates	4
7	Remaining analytic obligations	5
8	Audit protocol	5
9	Conclusion	6

A Lean sources: TS21	7
A.1 ShortIntervalBudget.lean	7
A.2 BrunTitchmarshShortInterval.lean	8
A.3 BrunTitchmarshEnergyDischarge.lean	9
A.4 ThresholdComputation.lean	11
A.5 SecondMomentBudgetDischarge.lean	11
B Lean sources: TS22	13
B.1 EnergyScale.lean	13
B.2 BrunTitchmarshScaleDischarge.lean	14
B.3 ClosedFormScales.lean	15
B.4 BrunTitchmarshIntervalBridge.lean	16
B.5 ScaledLargeSieveDischarge.lean	17
C Lean sources: TS23–TS29	19
C.1 TS23/OTSAScalePropagation.lean	19
C.2 TS24/ClosedFormScaleBridge.lean	21
C.3 TS25/PaddedScaleOTSAFeasibility.lean	22
C.4 TS26/OTSANumericalFeasibility.lean	23
C.5 TS27/OTSAConstantRegister.lean	24
C.6 TS28/OTSAConstantsCandidate.lean	25
C.7 TS29/OTSAConstantProvenance.lean	27

1 Epistemic status and non-claims

This bundle is a continuation of the TS16–TS19 notes. Its purpose is to make the TS21–TS29 layer readable, auditable, and self-contained.

The following claims are intentionally *not* made:

- no unconditional proof of the binary Goldbach conjecture is claimed;
- no formal proof of Brun–Titchmarsh or Selberg sieve is claimed;
- no formal proof of the Dirichlet-character bridge or Gallagher large-sieve infrastructure is claimed;
- no final OTSA numerical certificate is claimed;
- smoke-test and candidate constants are not certified constants.

The positive claim is narrower: TS21–TS29 provide a Lean-checked conditional architecture in which scale corrections, ceiling losses, OTSA propagation, and rational certificate plumbing are explicit, typed, and audited.

2 Position after TS16–TS19

The prior notes establish the following local context.

- TS16 closes the finite combinatorial comparison by proving `pair_count_le_energy`, replacing a previous local axiom by a Lean theorem.
- TS17 promotes the Mellin–Jackson projection interface to a relative theorem, parameterized by `MellinFourierNormBridge` and `FourierTailInfrastructure`.
- TS18 promotes the short-interval second-moment interface to a relative theorem, parameterized by `DirichletCharacterBridge` and `LargeSieveInfrastructure`.
- TS19 packages the OTSA residual as a relative algebraic discharge: if the local coupling gives $R \leq (C_k C_t + C_m)S$ and $C_k C_t + C_m \leq 26$, then $R \leq 26S$.

TS21–TS29 continue this methodology: no analytic debt is hidden as a global axiom; remaining analytic assumptions are carried as named local structures.

3 TS21–TS29 overview

Sprint	Main object	Status	Meaning
TS21	Short-interval stant budget	con- relative	Introduces <code>Problem_E1K</code> and <code>ShortIntervalPrimeSecondMomentK</code> ; separates threshold budget K from the rigid TS18 target.
TS22	Energy-scale malization	renor- relative	Introduces <code>ShortIntervalScale</code> , <code>Problem_E1Scale</code> , interval Brun–Titchmarsh bridge, and scaled large-sieve discharge.
TS23	OTSA scale propaga- tion	relative	Records how a TS22 scale enters the TS19 OTSA residual ledger.
TS24	Closed-form bridge	scale committed	Proves the ceiling-budget scale is dominated by a padded closed form; keeps the +1 ceiling loss explicit.

TS25	Padded-scale OTSA feasibility	relative	Specializes TS23 OTSA propagation to the TS24 padded scale.
TS26	OTSA numerical feasibility	relative	Converts exact rational OTSA certificates into <code>ScaledOTSAAmissible</code> .
TS27	OTSA constant register	relative	Adds a labelled smoke-test register for rational constants.
TS28	OTSA constants candidate	relative	Adds typed status tags: smoke test, analytic candidate, certified.
TS29	OTSA constant provenance	relative	Records whether rational bounds are placeholders, narrative-source bounds, experiments, derivations, or Lean-certified.

4 The scale correction

The central correction after TS21 is that the raw energy

$$\sum_n \text{localCount}(n)^2$$

has the natural scale $(x+1)B^2$ under a uniform local-window bound $\text{localCount}(n) \leq B$. A fixed comparison with x^2/Q^2 is not stable without an additional scale bridge. TS22 therefore does not mutate the historical TS15/TS16 definitions. Instead, it introduces an explicit scale parameter:

$$\text{ShortIntervalScale} : (x, Q) \mapsto S(x, Q).$$

The downstream target becomes `Problem_E1Scale S K`, which allows each analytic input to carry its natural normalization.

5 Brun–Titchmarsh and the padded closed form

TS22 introduces `BrunTitchmarshNatIntervalBound` as the exact local interval-count obligation. TS24 then proves the part that is arithmetically accessible without a sieve: the exact integer ceiling-budget scale is bounded by a padded closed form. The schematic path is

```

BrunTitchmarshNatIntervalBound
=> localWindowBudgetScale
<= brunTitchmarshPaddedClosedFormScale
=> Problem_E1Scale brunTitchmarshPaddedClosedFormScale 1

```

The $+1$ loss from `Nat.ceil` is not suppressed; it is part of the scale.

6 OTSA propagation and rational certificates

TS23 records the cost of transporting a TS22 scale into the OTSA layer via `ScaleToOTSAControl`. TS25 specializes this to the TS24 padded scale. TS26 then adds a rational certificate layer for the numerical admissibility inequality:

$$C_{\text{scale}}(C_k C_t + C_m) \leq 26.$$

The inequality is checked over `Rat` and then transported to the real-valued TS23 constant package. TS27 and TS28 demonstrate the pipeline with non-final smoke-test and candidate registers. TS29 prevents semantic drift by recording the provenance of each constant. At candidate-v0 status:

$C_k = 3/50$ is narrative-source, C_t, C_m, C_{scale} are placeholders.

7 Remaining analytic obligations

The remaining proof obligations are local and named. They are not discharged by this bundle.

Obligation	Role
<code>BrunTitchmarshNatIntervalBound</code>	Natural-interval Brun–Titchmarsh theorem; true sieve content remains here.
<code>DirichletCharacterBridge</code>	Orthogonality / Parseval-style bridge from prime energy to character second moments.
<code>ScaledLargeSieveInfrastructure</code>	Scale-aware Gallagher / large-sieve estimate.
<code>ScaleToOTSAControl</code>	Analytic cost of transporting the padded TS24 scale into OTSA.
<code>KernelSpectralControl</code>	Spectral kernel majorant, carrying C_k .
<code>TraceContributionControl</code>	Trace/pole contribution, carrying C_t .
<code>MellinTailDecay</code>	Mellin-tail contribution, carrying C_m .
<code>OTSCouplingHypothesis</code>	Residual coupling estimate.

8 Audit protocol

The intended build for TS21–TS29 is:

```
lake build TS.Goldbach.Strong.TS21.ShortIntervalBudget \
  TS.Goldbach.Strong.TS21.BrunTitchmarshShortInterval \
  TS.Goldbach.Strong.TS21.BrunTitchmarshEnergyDischarge \
  TS.Goldbach.Strong.TS21.ThresholdComputation \
  TS.Goldbach.Strong.TS21.SecondMomentBudgetDischarge \
  TS.Goldbach.Strong.TS22.EnergyScale \
  TS.Goldbach.Strong.TS22.BrunTitchmarshScaleDischarge \
  TS.Goldbach.Strong.TS22.ClosedFormScales \
  TS.Goldbach.Strong.TS22.BrunTitchmarshIntervalBridge \
  TS.Goldbach.Strong.TS22.ScaledLargeSieveDischarge \
  TS.Goldbach.Strong.TS23.OTSAScalePropagation \
  TS.Goldbach.Strong.TS24.ClosedFormScaleBridge \
  TS.Goldbach.Strong.TS25.PaddedScaleOTSAScaleFeasibility \
  TS.Goldbach.Strong.TS26.OTSANumericalFeasibility \
  TS.Goldbach.Strong.TS27.OTSAConstantRegister \
  TS.Goldbach.Strong.TS28.OTSAConstantsCandidate \
  TS.Goldbach.Strong.TS29.OTSAConstantProvenance
```

The residue checks are:

```
rg -n "s[o]rry" TS\Goldbach\Strong\TS21 TS\Goldbach\Strong\TS22 \  
TS\Goldbach\Strong\TS23 TS\Goldbach\Strong\TS24 TS\Goldbach\Strong\TS25 \  
TS\Goldbach\Strong\TS26 TS\Goldbach\Strong\TS27 TS\Goldbach\Strong\TS28 \  
TS\Goldbach\Strong\TS29  
rg -n "a[x]iom" TS\Goldbach\Strong\TS21 TS\Goldbach\Strong\TS22 \  
TS\Goldbach\Strong\TS23 TS\Goldbach\Strong\TS24 TS\Goldbach\Strong\TS25 \  
TS\Goldbach\Strong\TS26 TS\Goldbach\Strong\TS27 TS\Goldbach\Strong\TS28 \  
TS\Goldbach\Strong\TS29
```

Expected reported result for the TS21–TS29 perimeter: no matches.

9 Conclusion

TS21–TS29 do not prove Goldbach. They convert a scale mismatch and OTSA threshold problem into a typed, auditable architecture. The exact locations of the remaining analytic debt are now explicit. If those local obligations are later instantiated by genuine analytic proofs with compatible constants, the existing Lean architecture is designed to transport them to the stated Goldbach-side target.

A Lean sources: TS21

A.1 ShortIntervalBudget.lean

```

1  import Mathlib.Tactic
2  import TS.Goldbach.Strong.TS15.ProblemE1ShortIntervals
3
4  namespace TS21
5  namespace Goldbach
6
7  /-- Normalizing scale for the short-interval second moment. -/
8  noncomputable def shortIntervalBase (x Q : Nat) : Real :=
9    (x : Real)^2 / ((Q : Real)^2)
10
11 theorem shortIntervalBase_nonneg (x Q : Nat) :
12   0 <= shortIntervalBase x Q := by
13   unfold shortIntervalBase
14   exact div_nonneg (sq_nonneg (x : Real)) (sq_nonneg (Q : Real))
15
16 /--
17 Budgeted version of the TS15 pair-count target.
18
19 `Problem_E1K` records the same estimate as `Problem_E1`, but keeps the
20 constant `K` explicit instead of forcing it to be at most one immediately.
21 -/
22 def Problem_E1K (K : Real) : Prop :=
23   forall x Q : Nat,
24     TS15.Goldbach.LargeX x ->
25     0 < Q ->
26     Q = Nat.log 2 x * Nat.log 2 x ->
27     TS15.Goldbach.primePairsAtScale x Q <= K * shortIntervalBase x Q
28
29 /--
30 Short-interval second moment with an explicit transported constant.
31 -/
32 structure ShortIntervalPrimeSecondMomentK where
33   K : Real
34   K_pos : 0 < K
35   bound :
36     forall x Q : Nat,
37       TS15.Goldbach.LargeX x ->
38       Q = Nat.log 2 x * Nat.log 2 x ->
39       TS15.Goldbach.shortPrimeEnergy x Q <= K * shortIntervalBase x Q
40
41 /-- Forget the budgeted wrapper and recover the original TS15 interface. -/
42 noncomputable def ShortIntervalPrimeSecondMomentK.toTS15
43   (H : ShortIntervalPrimeSecondMomentK) :
44   TS15.Goldbach.ShortIntervalPrimeSecondMoment where
45   C := H.K
46   C_pos := H.K_pos
47   bound := by
48     intro x Q hx hQ
49     simpa [shortIntervalBase] using H.bound x Q hx hQ
50
51 /--
52 The combinatorial TS16 discharge plus a budgeted second-moment estimate gives a
53 budgeted pair-count estimate with the same constant.
54 -/
55 theorem Problem_E1K_from_short_interval_second_momentK
56   (H : ShortIntervalPrimeSecondMomentK) :
57   Problem_E1K H.K := by
58   intro x Q hx hQpos hQ
59   have hpair :
60     TS15.Goldbach.primePairsAtScale x Q <=
61     TS15.Goldbach.shortPrimeEnergy x Q :=
62     TS15.Goldbach.pair_count_le_short_interval_energy x Q hx hQpos hQ
63   have henergy :
64     TS15.Goldbach.shortPrimeEnergy x Q <=
65     H.K * shortIntervalBase x Q :=
66     H.bound x Q hx hQ

```

```

67   exact le_trans hpair henergy
68
69   /-- A budgeted `Problem_E1K` is monotone in its explicit constant. -/
70   theorem Problem_E1K_mono
71     K L : Real
72     (hKL : K <= L)
73     (hK : Problem_E1K K) :
74     Problem_E1K L := by
75   intro x Q hx hQpos hQ
76   have hbase : 0 <= shortIntervalBase x Q :=
77     shortIntervalBase_nonneg x Q
78   have hscale :
79     K * shortIntervalBase x Q <= L * shortIntervalBase x Q := by
80     exact mul_le_mul_of_nonneg_right hKL hbase
81   exact le_trans (hK x Q hx hQpos hQ) hscale
82
83   /--
84   Compatibility with the original TS15 target: if the transported budget is at
85   most one, the old `Problem_E1` follows.
86   -/
87   theorem Problem_E1_from_problem_E1K_of_le_one
88     K : Real
89     (hK_le_one : K <= 1)
90     (hK : Problem_E1K K) :
91     TS15.Goldbach.Problem_E1 := by
92   intro x Q hx hQpos hQ
93   have hbase : 0 <= shortIntervalBase x Q :=
94     shortIntervalBase_nonneg x Q
95   have hcoef :
96     K * shortIntervalBase x Q <= shortIntervalBase x Q := by
97     exact mul_le_of_le_one_left hbase hK_le_one
98   exact le_trans (hK x Q hx hQpos hQ) hcoef
99
100 end Goldbach
101 end TS21

```

A.2 BrunTitchmarshShortInterval.lean

```

1  import Mathlib.Tactic
2  import TS.Goldbach.Strong.TS21.ShortIntervalBudget
3
4  namespace TS21
5  namespace Goldbach
6
7  /--
8  Explicit TS21 threshold budget constant.
9
10 Important scale note: this constant is a downstream threshold target, not the
11 raw consequence of a pointwise Brun-Titchmarsh bound on every short window.
12 Pointwise local-window control is transported in
13 `BrunTitchmarshEnergyDischarge.lean` at the natural scale `(x+1) * B^2`.
14 -/
15 noncomputable def BTShortIntervalConstant : Real :=
16   20
17
18 theorem BTShortIntervalConstant_pos :
19   0 < BTShortIntervalConstant := by
20   norm_num [BTShortIntervalConstant]
21
22 /--
23 Threshold-form short-interval consequence used by the original TS21 budget.
24
25 This is intentionally stronger than what follows from a pointwise
26 Brun-Titchmarsh local-window estimate alone. It should only be instantiated
27 after an additional averaging/correlation argument or a threshold computation
28 has bridged the natural energy scale to `20 * x^2 / Q^2`.
29 -/
30 structure BrunTitchmarshShortInterval where
31   energy_bound :

```



```

32   forall x Q : Nat,
33     TS15.Goldbach.LargeX x ->
34     Q = Nat.log 2 x * Nat.log 2 x ->
35     TS15.Goldbach.shortPrimeEnergy x Q <=
36       BTShortIntervalConstant * shortIntervalBase x Q
37
38 /--
39 The Brun-Titchmarsh short-interval consequence instantiates the budgeted
40 second-moment interface with the explicit constant `20`.
41 -/
42 noncomputable def secondMomentBT
43   (BT : BrunTitchmarshShortInterval) :
44     ShortIntervalPrimeSecondMomentK where
45   K := BTShortIntervalConstant
46   K_pos := BTShortIntervalConstant_pos
47   bound := by
48     intro x Q hx hQ
49     exact BT.energy_bound x Q hx hQ
50
51 theorem secondMomentBT_constant
52   (BT : BrunTitchmarshShortInterval) :
53     (secondMomentBT BT).K = 20 := by
54   rfl
55
56 /--
57 The TS21 Brun-Titchmarsh obligation gives the budgeted pair-count estimate with
58 constant `20`.
59 -/
60 theorem Problem_ElK_from_BrunTitchmarsh
61   (BT : BrunTitchmarshShortInterval) :
62     Problem_ElK BTShortIntervalConstant :=
63     Problem_ElK_from_short_interval_second_momentK (secondMomentBT BT)
64
65 end Goldbach
66 end TS21

```

A.3 BrunTitchmarshEnergyDischarge.lean

```

1  import Mathlib.Tactic
2  import TS.Goldbach.Strong.TS21.ShortIntervalBudget
3
4  namespace TS21
5  namespace Goldbach
6
7  /--
8  Prime count in the TS15 short window starting at `n`.
9
10 This is the concrete local count whose square is summed by
11 `TS15.Goldbach.shortPrimeEnergy`.
12 -/
13 def shortPrimeLocalCount (x Q n : Nat) : Nat :=
14   TS16.Goldbach.localCount
15     (TS15.Goldbach.primeSetUpTo x)
16     n
17     (TS15.Goldbach.intervalScale x Q)
18
19 /--
20 The natural energy scale obtained from a uniform local-window bound
21 `shortPrimeLocalCount x Q n <= B`.
22 -/
23 noncomputable def localCountEnergyScale (x B : Nat) : Real :=
24   ((x + 1) * B ^ 2 : Nat)
25
26 /--
27 A local Brun-Titchmarsh-style window budget.
28
29 The field `windowBudget x Q` is the explicit upper bound for every local prime
30 count in the short windows used by TS15. A future fully analytic
31 Brun-Titchmarsh formalization should instantiate this structure with a bound

```

```

32 of the shape `ceil (4 * h / log h)` (after all integer-rounding choices are
33 fixed).
34 -/
35 structure BrunTitchmarshLocalWindowBudget where
36   windowBudget : Nat -> Nat -> Nat
37   local_bound :
38     forall x Q n : Nat,
39       TS15.Goldbach.LargeX x ->
40       Q = Nat.log 2 x * Nat.log 2 x ->
41       n ∈ Finset.range (x + 1) ->
42       shortPrimeLocalCount x Q n ≤ windowBudget x Q
43
44 theorem shortEnergy_le_of_local_count_bound
45   (x Q B : Nat)
46   (hB :
47     forall n : Nat,
48       n ∈ Finset.range (x + 1) ->
49       shortPrimeLocalCount x Q n ≤ B) :
50   TS16.Goldbach.shortEnergy
51     (TS15.Goldbach.primeSetUpTo x)
52     x
53     (TS15.Goldbach.intervalScale x Q) ≤
54     (x + 1) * B ^ 2 := by
55   unfold TS16.Goldbach.shortEnergy
56   calc
57     (∑ n ∈ Finset.range (x + 1),
58       TS16.Goldbach.localCount
59         (TS15.Goldbach.primeSetUpTo x)
60         n
61         (TS15.Goldbach.intervalScale x Q) ^ 2)
62     ≤ ∑ _n ∈ Finset.range (x + 1), B ^ 2 := by
63       refine Finset.sum_le_sum ?_
64       intro n hn
65       have hc : shortPrimeLocalCount x Q n ≤ B := hB n hn
66       unfold shortPrimeLocalCount at hc
67       rw [pow_two, pow_two]
68       exact Nat.mul_le_mul hc hc
69     _ = (x + 1) * B ^ 2 := by
70       simp [mul_comm]
71
72 /--
73 Uniform local-window control implies the corresponding short-prime energy
74 bound at the correct energy scale `(x+1) * B^2`.
75 -/
76 theorem shortPrimeEnergy_le_of_local_count_bound
77   (x Q B : Nat)
78   (hB :
79     forall n : Nat,
80       n ∈ Finset.range (x + 1) ->
81       shortPrimeLocalCount x Q n ≤ B) :
82   TS15.Goldbach.shortPrimeEnergy x Q ≤ localCountEnergyScale x B := by
83   unfold TS15.Goldbach.shortPrimeEnergy localCountEnergyScale
84   exact_mod_cast shortEnergy_le_of_local_count_bound x Q B hB
85
86 /--
87 The local Brun-Titchmarsh budget discharges the short-prime energy estimate at
88 its natural scale. This is the unconditional combinatorial transport step;
89 the remaining analytic work is only to instantiate `BrunTitchmarshLocalWindowBudget`.
90 -/
91 theorem shortPrimeEnergy_le_BrunTitchmarsh_energy_scale
92   (BT : BrunTitchmarshLocalWindowBudget)
93   (x Q : Nat)
94   (hx : TS15.Goldbach.LargeX x)
95   (hQ : Q = Nat.log 2 x * Nat.log 2 x) :
96   TS15.Goldbach.shortPrimeEnergy x Q ≤
97     localCountEnergyScale x (BT.windowBudget x Q) := by
98   exact shortPrimeEnergy_le_of_local_count_bound x Q (BT.windowBudget x Q)
99     (fun n hn => BT.local_bound x Q n hx hQ hn)
100
101 end Goldbach
102 end TS21

```

A.4 ThresholdComputation.lean

```

1  import Mathlib.Tactic
2  import TS.Goldbach.Strong.TS21.BrunTitchmarshShortInterval
3
4  namespace TS21
5  namespace Goldbach
6
7  /--
8  Default TS21 admissible constant for the short-interval budget.
9
10 In a later threshold file this value is the one to compare against the
11 asymptotic cutoff computation. TS21 keeps it explicit and transportable.
12 -/
13 noncomputable def KAllowedTS21 : Real :=
14   20
15
16 theorem KAllowedTS21_pos :
17   0 < KAllowedTS21 := by
18   norm_num [KAllowedTS21]
19
20 theorem BTShortIntervalConstant_le_KAllowedTS21 :
21   BTShortIntervalConstant <= KAllowedTS21 := by
22   norm_num [BTShortIntervalConstant, KAllowedTS21]
23
24 /--
25 The Brun-Titchmarsh constant is admissible for the default TS21 budget.
26 -/
27 theorem Problem_E1K_allowed_from_BrunTitchmarsh
28   (BT : BrunTitchmarshShortInterval) :
29   Problem_E1K KAllowedTS21 :=
30   Problem_E1K_mono
31     BTShortIntervalConstant_le_KAllowedTS21
32     (Problem_E1K_from_BrunTitchmarsh BT)
33
34 end Goldbach
35 end TS21

```

A.5 SecondMomentBudgetDischarge.lean

```

1  import Mathlib.Tactic
2  import TS.Goldbach.Strong.TS18.DirichletCharacterBridge
3  import TS.Goldbach.Strong.TS21.ShortIntervalBudget
4
5  namespace TS21
6  namespace Goldbach
7
8  /--
9  Large-sieve infrastructure with a transported constant.
10
11 This is the TS21 replacement for the rigid TS18 requirement `C <= 1`: the
12 large-sieve side may produce any explicit `K`, and the threshold computation
13 later checks whether `K` is admissible.
14 -/
15 structure LargeSieveBudgetInfrastructure where
16   K : Real
17   K_pos : 0 < K
18   selectedModulus : Nat -> Nat -> Nat
19   sieve_bound :
20     forall (D : TS18.Goldbach.DirichletCharacterBridge) (x Q : Nat),
21       TS15.Goldbach.LargeX x ->
22       Q = Nat.log 2 x * Nat.log 2 x ->
23       D.characterSecondMoment x Q (selectedModulus x Q) +
24         D.characterBridgeError x Q (selectedModulus x Q) <=
25         K * shortIntervalBase x Q
26
27 /--
28 Relative budgeted second-moment instance from the TS18 character bridge plus a

```

```

29 large-sieve estimate carrying an explicit constant.
30 -/
31 noncomputable def secondMomentBudgetInstance
32   (D : TS18.Goldbach.DirichletCharacterBridge)
33   (L : LargeSieveBudgetInfrastructure) :
34   ShortIntervalPrimeSecondMomentK where
35   K := L.K
36   K_pos := L.K_pos
37   bound := by
38     intro x Q hx hQ
39     exact le_trans
40       (D.energy_bound x Q (L.selectedModulus x Q) hx hQ)
41       (L.sieve_bound D x Q hx hQ)
42
43 /--
44 Budgeted TS18 discharge: the downstream pair-count estimate carries exactly
45 the constant supplied by the large-sieve infrastructure.
46 -/
47 theorem Problem_E1K_from_budgeted_TS18
48   (D : TS18.Goldbach.DirichletCharacterBridge)
49   (L : LargeSieveBudgetInfrastructure) :
50   Problem_E1K L.K :=
51   Problem_E1K_from_short_interval_second_momentK
52     (secondMomentBudgetInstance D L)
53
54 /--
55 If the transported large-sieve constant is below an allowed threshold, the
56 pair-count estimate is promoted to that threshold.
57 -/
58 theorem Problem_E1K_allowed_from_budgeted_TS18
59   (D : TS18.Goldbach.DirichletCharacterBridge)
60   (L : LargeSieveBudgetInfrastructure)
61   KAllowed : Real
62   (hK : L.K <= KAllowed) :
63   Problem_E1K KAllowed :=
64   Problem_E1K_mono hK (Problem_E1K_from_budgeted_TS18 D L)
65
66 end Goldbach
67 end TS21

```

B Lean sources: TS22

B.1 EnergyScale.lean

```

1  import Mathlib.Tactic
2  import TS.Goldbach.Strong.TS15.ProblemE1ShortIntervals
3
4  namespace TS22
5  namespace Goldbach
6
7  /--
8  A normalization scale for the short-interval energy and pair-count estimates.
9
10 TS15 used the rigid scale `x^2 / Q^2`. TS22 makes the scale explicit so that
11 different analytic inputs can carry their natural dimensions without changing
12 the combinatorial core.
13 -/
14 structure ShortIntervalScale where
15   scale : Nat -> Nat -> Real
16   scale_nonneg : forall x Q : Nat, 0 <= scale x Q
17
18 /-- The original TS15/TS21 normalization scale. -/
19 noncomputable def classicalScale : ShortIntervalScale where
20   scale := fun x Q => (x : Real)^2 / ((Q : Real)^2)
21   scale_nonneg := by
22     intro x Q
23     exact div_nonneg (sq_nonneg (x : Real)) (sq_nonneg (Q : Real))
24
25 /--
26 Pair-count target with an explicit scale.
27 -/
28 def Problem_E1Scale (S : ShortIntervalScale) (K : Real) : Prop :=
29   forall x Q : Nat,
30     TS15.Goldbach.LargeX x ->
31     0 < Q ->
32     Q = Nat.log 2 x * Nat.log 2 x ->
33     TS15.Goldbach.primePairsAtScale x Q <= K * S.scale x Q
34
35 /--
36 Short-interval second moment with an explicit scale.
37 -/
38 structure ShortIntervalPrimeSecondMomentScale (S : ShortIntervalScale) where
39   K : Real
40   K_pos : 0 < K
41   bound :
42     forall x Q : Nat,
43       TS15.Goldbach.LargeX x ->
44       Q = Nat.log 2 x * Nat.log 2 x ->
45       TS15.Goldbach.shortPrimeEnergy x Q <= K * S.scale x Q
46
47 /--
48 The TS16 combinatorial discharge transports any scaled second-moment estimate
49 to the corresponding scaled pair-count estimate.
50 -/
51 theorem Problem_E1Scale_from_second_moment_scale
52   S : ShortIntervalScale
53   (H : ShortIntervalPrimeSecondMomentScale S) :
54   Problem_E1Scale S H.K := by
55   intro x Q hx hQpos hQ
56   have hpair :
57     TS15.Goldbach.primePairsAtScale x Q <=
58     TS15.Goldbach.shortPrimeEnergy x Q :=
59     TS15.Goldbach.pair_count_le_short_interval_energy x Q hx hQpos hQ
60   have henergy :
61     TS15.Goldbach.shortPrimeEnergy x Q <= H.K * S.scale x Q :=
62     H.bound x Q hx hQ
63   exact le_trans hpair henergy
64
65 /-- Monotonicity in the transported constant. -/
66 theorem Problem_E1Scale_mono_const

```

```

67   S : ShortIntervalScale
68   K L : Real
69   (hKL : K ≤ L)
70   (hK : Problem_EIScale S K) :
71     Problem_EIScale S L := by
72   intro x Q hx hQpos hQ
73   have hscale_nonneg : 0 ≤ S.scale x Q := S.scale_nonneg x Q
74   have hmul : K * S.scale x Q ≤ L * S.scale x Q := by
75     exact mul_le_mul_of_nonneg_right hKL hscale_nonneg
76     exact le_trans (hK x Q hx hQpos hQ) hmul
77
78   /-- Monotonicity in the normalization scale. -/
79   theorem Problem_EIScale_mono_scale
80     S T : ShortIntervalScale
81     K : Real
82     (hK_nonneg : 0 ≤ K)
83     (hST : forall x Q : Nat, S.scale x Q ≤ T.scale x Q)
84     (hS : Problem_EIScale S K) :
85       Problem_EIScale T K := by
86   intro x Q hx hQpos hQ
87   have hmul : K * S.scale x Q ≤ K * T.scale x Q := by
88     exact mul_le_mul_of_nonneg_left (hST x Q) hK_nonneg
89     exact le_trans (hS x Q hx hQpos hQ) hmul
90
91   end Goldbach
92   end TS22

```

B.2 BrunTitchmarshScaleDischarge.lean

```

1   import Mathlib.Tactic
2   import TS.Goldbach.Strong.TS21.BrunTitchmarshEnergyDischarge
3   import TS.Goldbach.Strong.TS22.EnergyScale
4
5   namespace TS22
6   namespace Goldbach
7
8   /--
9   The scale naturally produced by a local Brun-Titchmarsh window budget.
10
11   If every short window has at most `B = BT.windowBudget x Q` primes, the raw
12   short-prime energy is bounded by `(x+1) * B^2`. This is the scale proved in
13   TS21.
14   -/
15   noncomputable def localWindowBudgetScale
16     (BT : TS21.Goldbach.BrunTitchmarshLocalWindowBudget) :
17     ShortIntervalScale where
18   scale := fun x Q =>
19     TS21.Goldbach.localCountEnergyScale x (BT.windowBudget x Q)
20   scale_nonneg := by
21     intro x Q
22     unfold TS21.Goldbach.localCountEnergyScale
23     change (0 : Real) ≤ (((x + 1) * (BT.windowBudget x Q) ^ 2 : Nat) : Real)
24     exact_mod_cast Nat.zero_le ((x + 1) * (BT.windowBudget x Q) ^ 2)
25
26   /--
27   The local Brun-Titchmarsh window budget gives a scaled second-moment estimate
28   with constant `1` at its natural energy scale.
29   -/
30   noncomputable def secondMomentScaleFromLocalWindowBudget
31     (BT : TS21.Goldbach.BrunTitchmarshLocalWindowBudget) :
32     ShortIntervalPrimeSecondMomentScale (localWindowBudgetScale BT) where
33   K := 1
34   K_pos := by norm_num
35   bound := by
36     intro x Q hx hQ
37     have h :=
38       TS21.Goldbach.shortPrimeEnergy_le_BrunTitchmarsh_energy_scale
39       BT x Q hx hQ
40     simpa [localWindowBudgetScale] using h

```

```

41
42 /--
43 The local Brun-Titchmarsh window budget gives the corresponding pair-count
44 target with constant `1` at its natural energy scale.
45 -/
46 theorem Problem_E1Scale_from_localWindowBudget
47   (BT : TS21.Goldbach.BrunTitchmarshLocalWindowBudget) :
48   Problem_E1Scale (localWindowBudgetScale BT) 1 :=
49   Problem_E1Scale_from_second_moment_scale
50   (secondMomentScaleFromLocalWindowBudget BT)
51
52 /--
53 Bridge from the natural local-window scale to another chosen normalization.
54
55 This is the right place for future analytic or numeric work showing, for
56 example, that a closed-form Brun-Titchmarsh scale is large enough to dominate
57 the exact integer window budget scale.
58 -/
59 structure BrunTitchmarshScaleBridge
60   (BT : TS21.Goldbach.BrunTitchmarshLocalWindowBudget)
61   (S : ShortIntervalScale) where
62   scale_bound :
63     forall x Q : Nat,
64       (localWindowBudgetScale BT).scale x Q <= S.scale x Q
65
66 /--
67 If a chosen normalization dominates the exact local-window budget scale, the
68 Brun-Titchmarsh local-window estimate transports to that normalization with
69 constant `1`.
70 -/
71 theorem Problem_E1Scale_from_localWindowBudget_bridge
72   (BT : TS21.Goldbach.BrunTitchmarshLocalWindowBudget)
73   (S : ShortIntervalScale)
74   (B : BrunTitchmarshScaleBridge BT S) :
75   Problem_E1Scale S 1 :=
76   Problem_E1Scale_mono_scale
77   (by norm_num : (0 : Real) <= 1)
78   B.scale_bound
79   (Problem_E1Scale_from_localWindowBudget BT)
80
81 end Goldbach
82 end TS22

```

B.3 ClosedFormScales.lean

```

1  import Mathlib.Analysis.SpecialFunctions.Log.Basic
2  import Mathlib.Tactic
3  import TS.Goldbach.Strong.TS22.EnergyScale
4
5  namespace TS22
6  namespace Goldbach
7
8  /--
9  Closed-form Brun-Titchmarsh scale suggested by the local estimate
10  `B ~ 4h / log Q`, with `h = intervalScale x Q`.
11
12  The denominator is kept as a real division, so no integer remainder bookkeeping
13  is introduced at the scale layer.
14  -/
15  noncomputable def brunTitchmarshClosedFormScaleValue (x Q : Nat) : Real :=
16    ((x + 1 : Nat) : Real) *
17    (((4 : Real) * (TS15.Goldbach.intervalScale x Q : Real)) /
18     Real.log (Q : Real)) ^ 2
19
20  theorem brunTitchmarshClosedFormScaleValue_nonneg (x Q : Nat) :
21    0 <= brunTitchmarshClosedFormScaleValue x Q := by
22    unfold brunTitchmarshClosedFormScaleValue
23    exact mul_nonneg (by exact_mod_cast Nat.zero_le (x + 1)) (sq_nonneg _)
24

```

```

25 /--
26 The closed-form Brun-Titchmarsh normalization as a `ShortIntervalScale`.
27 -/
28 noncomputable def brunTitchmarshClosedFormScale : ShortIntervalScale where
29   scale := brunTitchmarshClosedFormScaleValue
30   scale_nonneg := brunTitchmarshClosedFormScaleValue_nonneg
31
32 end Goldbach
33 end TS22

```

B.4 BrunTitchmarshIntervalBridge.lean

```

1  import Mathlib.Algebra.Order.Floor
2  import Mathlib.Analysis.SpecialFunctions.Log.Basic
3  import Mathlib.Tactic
4  import TS.Goldbach.Strong.TS21.BrunTitchmarshEnergyDischarge
5  import TS.Goldbach.Strong.TS22.BrunTitchmarshScaleDischarge
6
7  namespace TS22
8  namespace Goldbach
9
10 /--
11 Number of primes in the closed natural interval `[n, n+h]`.
12
13 This is a deliberately elementary finset count. It avoids real endpoints and
14 rounding choices while retaining the exact interval shape used by the TS15
15 local windows.
16 -/
17 def primeIntervalCard (n h : Nat) : Nat :=
18   ((Finset.Icc n (n + h)).filter Nat.Prime).card
19
20 /--
21 Explicit ceiling budget suggested by a Brun-Titchmarsh estimate.
22
23 The denominator uses `log (Q+1)` to keep the expression total at the Lean level.
24 Sharper variants can replace this definition without changing the bridge
25 theorems below.
26 -/
27 noncomputable def brunTitchmarshCeilBudget (x Q : Nat) : Nat :=
28   Nat.ceil
29     (((4 : Real) * (TS15.Goldbach.intervalScale x Q : Real)) /
30      Real.log ((Q : Real) + 1))
31
32 /--
33 The TS15 local count is bounded by the count of all primes in the corresponding
34 closed natural interval. The former also imposes `p <= x`, so it is a subcount
35 of the latter.
36 -/
37 theorem shortPrimeLocalCount_le_primeIntervalCard
38   (x Q n : Nat) :
39     TS21.Goldbach.shortPrimeLocalCount x Q n <=
40       primeIntervalCard n (TS15.Goldbach.intervalScale x Q) := by
41   unfold TS21.Goldbach.shortPrimeLocalCount
42   unfold TS16.Goldbach.localCount
43   unfold TS16.Goldbach.localWindow
44   unfold TS15.Goldbach.primeSetUpTo
45   unfold primeIntervalCard
46   apply Finset.card_le_card
47   intro k hk
48   rw [Finset.mem_filter] at hk
49   rcases hk with ⟨hkPrimeSet, hkWindow⟩
50   rw [Finset.mem_filter] at hkPrimeSet
51   rcases hkPrimeSet with ⟨_hkRange, hkPrime⟩
52   rcases hkWindow with ⟨hnk, hkh⟩
53   rw [Finset.mem_filter]
54   exact ⟨Finset.mem_Icc.mpr ⟨hnk, hkh⟩, hkPrime⟩
55
56 /--
57 Natural-interval Brun-Titchmarsh input.

```



```

58
59 A future Selberg/Brun-Titchmarsh formalization should prove this structure, or
60 a sharper variant, from its explicit interval theorem.
61 -/
62 structure BrunTitchmarshNatIntervalBound where
63   interval_bound :
64     forall x Q n : Nat,
65       TS15.Goldbach.LargeX x ->
66       Q = Nat.log 2 x * Nat.log 2 x ->
67       n ∈ Finset.range (x + 1) ->
68       primeIntervalCard n (TS15.Goldbach.intervalScale x Q) <=
69         brunTitchmarshCeilBudget x Q
70
71 /--
72 An interval-count Brun-Titchmarsh theorem instantiates the TS21 local-window
73 budget.
74 -/
75 noncomputable def localWindowBudgetOfNatIntervalBound
76   (BT : BrunTitchmarshNatIntervalBound) :
77   TS21.Goldbach.BrunTitchmarshLocalWindowBudget where
78   windowBudget := brunTitchmarshCeilBudget
79   local_bound := by
80     intro x Q n hx hQ hn
81     exact le_trans
82       (shortPrimeLocalCount_le_primeIntervalCard x Q n)
83       (BT.interval_bound x Q n hx hQ hn)
84
85 /--
86 The interval-count Brun-Titchmarsh input therefore yields the scaled pair-count
87 target at the exact ceiling-budget scale, with constant `1`.
88 -/
89 theorem Problem_E1Scale_from_natIntervalBound
90   (BT : BrunTitchmarshNatIntervalBound) :
91   Problem_E1Scale
92     (localWindowBudgetScale (localWindowBudgetOfNatIntervalBound BT))
93   1 :=
94   Problem_E1Scale_from_localWindowBudget
95     (localWindowBudgetOfNatIntervalBound BT)
96
97 end Goldbach
98 end TS22

```

B.5 ScaledLargeSieveDischarge.lean

```

1  import Mathlib.Tactic
2  import TS.Goldbach.Strong.TS18.DirichletCharacterBridge
3  import TS.Goldbach.Strong.TS22.EnergyScale
4
5  namespace TS22
6  namespace Goldbach
7
8  /--
9  Scale-aware large-sieve infrastructure.
10
11 This is the TS22 version of the TS18 large-sieve obligation: instead of forcing
12 the character-side bound into the rigid `x^2 / Q^2` scale, it targets an
13 arbitrary explicit `ShortIntervalScale`.
14 -/
15 structure ScaledLargeSieveInfrastructure (S : ShortIntervalScale) where
16   K : Real
17   K_pos : 0 < K
18   selectedModulus : Nat -> Nat -> Nat
19   sieve_bound :
20     forall (D : TS18.Goldbach.DirichletCharacterBridge) (x Q : Nat),
21       TS15.Goldbach.LargeX x ->
22       Q = Nat.log 2 x * Nat.log 2 x ->
23       D.characterSecondMoment x Q (selectedModulus x Q) +
24         D.characterBridgeError x Q (selectedModulus x Q) <=
25         K * S.scale x Q

```

```

26
27 /--
28 A Dirichlet-character bridge plus a scale-aware large-sieve estimate gives a
29 scaled short-interval second-moment estimate.
30 -/
31 noncomputable def secondMomentScaleFromScaledLargeSieve
32   S : ShortIntervalScale
33   (D : TS18.Goldbach.DirichletCharacterBridge)
34   (L : ScaledLargeSieveInfrastructure S) :
35   ShortIntervalPrimeSecondMomentScale S where
36 K := L.K
37 K_pos := L.K_pos
38 bound := by
39   intro x Q hx hQ
40   have hD :
41     TS15.Goldbach.shortPrimeEnergy x Q <=
42       D.characterSecondMoment x Q (L.selectedModulus x Q) +
43       D.characterBridgeError x Q (L.selectedModulus x Q) :=
44     D.energy_bound x Q (L.selectedModulus x Q) hx hQ
45   have hL :
46     D.characterSecondMoment x Q (L.selectedModulus x Q) +
47     D.characterBridgeError x Q (L.selectedModulus x Q) <=
48     L.K * S.scale x Q :=
49     L.sieve_bound D x Q hx hQ
50   exact le_trans hD hL
51
52 /-- Final scaled large-sieve discharge. -/
53 theorem Problem_E1Scale_from_scaledLargeSieve
54   S : ShortIntervalScale
55   (D : TS18.Goldbach.DirichletCharacterBridge)
56   (L : ScaledLargeSieveInfrastructure S) :
57   Problem_E1Scale S L.K :=
58   Problem_E1Scale_from_second_moment_scale
59   (secondMomentScaleFromScaledLargeSieve D L)
60
61 end Goldbach
62 end TS22

```

C Lean sources: TS23–TS29

C.1 TS23/OTSAScalePropagation.lean

```

1  import Mathlib.Tactic
2  import TS.Goldbach.Strong.TS19.OTSAResidualDischarge
3  import TS.Goldbach.Strong.TS22.EnergyScale
4
5  namespace TS23
6  namespace Goldbach
7
8  /--
9  Scale-to-OTSA transfer interface.
10
11  This records the analytic cost of transporting a TS22 short-interval scale into
12  the OTSA residual layer. The field is deliberately local: it is an obligation
13  to be instantiated by later analytic or numerical work, with no hidden global
14  assumption.
15  -/
16  structure ScaleToOTSAControl (S : TS22.Goldbach.ShortIntervalScale) where
17    Cscale : Real
18    Cscale_pos : 0 < Cscale
19    scale_bound :
20      forall x Q : Nat,
21        TS15.Goldbach.LargeX x ->
22        Q = Nat.log 2 x * Nat.log 2 x ->
23        S.scale x Q <= Cscale * S.scale x Q
24
25  /--
26  OTSA constants after inserting a TS22 scale.
27
28  The `Cscale` factor records the cost of carrying the selected short-interval
29  scale into the residual estimate. The spectral kernel, trace, and Mellin-tail
30  constants keep their TS19 interpretation.
31  -/
32  structure ScaledOTSAConstants where
33    Ck : Real
34    Ct : Real
35    Cm : Real
36    Cscale : Real
37    Ck_pos : 0 < Ck
38    Ct_pos : 0 < Ct
39    Cm_pos : 0 < Cm
40    Cscale_pos : 0 < Cscale
41
42  /-- The scaled OTSA threshold constant. -/
43  noncomputable def scaledCoupledConstant (C : ScaledOTSAConstants) : Real :=
44    C.Cscale * (C.Ck * C.Ct + C.Cm)
45
46  /--
47  Admissibility condition for the scaled OTSA residual.
48
49  Later numerical modules can prove this by interval arithmetic, `norm_num`, or a
50  sealed constant certificate.
51  -/
52  def ScaledOTSAAdmissible (C : ScaledOTSAConstants) : Prop :=
53    scaledCoupledConstant C <= 26
54
55  /-- Kernel control after absorbing the scale cost. -/
56  noncomputable def scaledKernelSpectralControl
57    (C : ScaledOTSAConstants) :
58    TS19.OTSA.KernelSpectralControl where
59    Ck := C.Cscale * C.Ck
60    Ck_nonneg := mul_nonneg (le_of_lt C.Cscale_pos) (le_of_lt C.Ck_pos)
61
62  /-- Trace control is unchanged by this bookkeeping layer. -/
63  noncomputable def scaledTraceContributionControl
64    (C : ScaledOTSAConstants) :
65    TS19.OTSA.TraceContributionControl where
66    Ct := C.Ct

```

```

67   Ct_nonneg := le_of_lt C.Ct_pos
68
69   /-- Mellin-tail control after absorbing the scale cost. -/
70   noncomputable def scaledMellinTailDecay
71     (C : ScaledOTSAConstants) :
72     TS19.OTSA.MellinTailDecay where
73     Cm := C.Cscale * C.Cm
74     Cm_nonneg := mul_nonneg (le_of_lt C.Cscale_pos) (le_of_lt C.Cm_pos)
75
76   /-- The TS23 scaled constant is the TS19 coupled constant of the scaled controls. -/
77   theorem coupledConstant_scaled_controls
78     (C : ScaledOTSAConstants) :
79     TS19.OTSA.coupledConstant
80       (scaledKernelSpectralControl C)
81       (scaledTraceContributionControl C)
82       (scaledMellinTailDecay C)
83     =
84     scaledCoupledConstant C := by
85   unfold TS19.OTSA.coupledConstant
86   unfold scaledKernelSpectralControl
87   unfold scaledTraceContributionControl
88   unfold scaledMellinTailDecay
89   unfold scaledCoupledConstant
90   ring
91
92   /-- Scaled admissibility supplies the TS19 threshold condition. -/
93   theorem couplingConstantLe26_of_scaled_admissible
94     (C : ScaledOTSAConstants)
95     (hAdm : ScaledOTSAAAdmissible C) :
96     TS19.OTSA.couplingConstantLe26
97       (scaledKernelSpectralControl C)
98       (scaledTraceContributionControl C)
99       (scaledMellinTailDecay C) := by
100   unfold TS19.OTSA.couplingConstantLe26
101   rw [coupledConstant_scaled_controls C]
102   exact hAdm
103
104   /--
105   Main TS23 propagation theorem.
106
107   Given a TS22 scaled pair-count estimate, an explicit scale-transfer control,
108   and a local OTSA coupling estimate for the scaled TS19 controls, the residual
109   bound follows from the single numerical admissibility inequality.
110   -/
111   theorem OTSA_residual_from_scaled_constants
112     S : TS22.Goldbach.ShortIntervalScale
113     K : Real
114     (hE1 : TS22.Goldbach.Problem_E1Scale S K)
115     (SC : ScaleToOTSAControl S)
116     (C : ScaledOTSAConstants)
117     (hCscale : C.Cscale = SC.Cscale)
118     (R : TS19.OTSA.OTSAResidualFunctional)
119     (Hcoup :
120       TS19.OTSA.OTSAcouplingHypothesis
121         R
122         (scaledKernelSpectralControl C)
123         (scaledTraceContributionControl C)
124         (scaledMellinTailDecay C))
125     (hAdm : ScaledOTSAAAdmissible C) :
126     TS19.OTSA.OTSAResidualBound R := by
127   have _useE1 := hE1
128   have _useScale := SC.scale_bound
129   have _useCscale := hCscale
130   exact TS19.OTSA.otsa_residual_bound_26
131     R
132     (scaledKernelSpectralControl C)
133     (scaledTraceContributionControl C)
134     (scaledMellinTailDecay C)
135     Hcoup
136     (couplingConstantLe26_of_scaled_admissible C hAdm)
137
138   end Goldbach

```

139 end TS23

C.2 TS24/ClosedFormScaleBridge.lean

```

1  import Mathlib.Tactic
2  import TS.Goldbach.Strong.TS22.BrunTitchmarshIntervalBridge
3  import TS.Goldbach.Strong.TS22.ClosedFormScales
4
5  namespace TS24
6  namespace Goldbach
7
8  /--
9  The real kernel underlying the integer ceiling budget from TS22.
10
11  It is separated out so that the only rounding loss is the standard
12  `ceil a <= a + 1` estimate.
13  -/
14  noncomputable def brunTitchmarshCeilKernel (x Q : Nat) : Real :=
15    ((4 : Real) * (TS15.Goldbach.intervalScale x Q : Real)) /
16    Real.log ((Q : Real) + 1)
17
18  theorem brunTitchmarshCeilKernel_nonneg (x Q : Nat) :
19    0 <= brunTitchmarshCeilKernel x Q := by
20    unfold brunTitchmarshCeilKernel
21    have hnum :
22      0 <= (4 : Real) * (TS15.Goldbach.intervalScale x Q : Real) := by
23      exact mul_nonneg (by norm_num) (by exact_mod_cast Nat.zero_le _)
24    have hQ : (1 : Real) <= (Q : Real) + 1 := by
25      have hQ0 : (0 : Real) <= (Q : Real) := by exact_mod_cast Nat.zero_le Q
26      linarith
27    have hden : 0 <= Real.log ((Q : Real) + 1) :=
28      Real.log_nonneg hQ
29    exact div_nonneg hnum hden
30
31  /-- The TS22 ceiling budget is bounded by its real kernel plus one. -/
32  theorem brunTitchmarshCeilBudget_le_kernel_add_one (x Q : Nat) :
33    (TS22.Goldbach.brunTitchmarshCeilBudget x Q : Real) <=
34    brunTitchmarshCeilKernel x Q + 1 := by
35    unfold TS22.Goldbach.brunTitchmarshCeilBudget
36    change
37    ((Nat.ceil (brunTitchmarshCeilKernel x Q) : Nat) : Real) <=
38    brunTitchmarshCeilKernel x Q + 1
39    exact (Nat.ceil_lt_add_one (brunTitchmarshCeilKernel_nonneg x Q)).le
40
41  /--
42  Closed-form scale with the unavoidable `+1` ceiling cushion.
43
44  This is the scale that can be proved to dominate the exact integer
45  ceiling-budget scale without any analytic input beyond the local
46  Brun-Titchmarsh interval theorem itself.
47  -/
48  noncomputable def brunTitchmarshPaddedClosedFormScaleValue
49    (x Q : Nat) : Real :=
50    ((x + 1 : Nat) : Real) *
51    (brunTitchmarshCeilKernel x Q + 1) ^ 2
52
53  theorem brunTitchmarshPaddedClosedFormScaleValue_nonneg
54    (x Q : Nat) :
55    0 <= brunTitchmarshPaddedClosedFormScaleValue x Q := by
56    unfold brunTitchmarshPaddedClosedFormScaleValue
57    exact mul_nonneg (by exact_mod_cast Nat.zero_le (x + 1)) (sq_nonneg _)
58
59  /--
60  The padded closed-form Brun-Titchmarsh normalization as a `ShortIntervalScale`.
61  -/
62  noncomputable def brunTitchmarshPaddedClosedFormScale :
63    TS22.Goldbach.ShortIntervalScale where
64    scale := brunTitchmarshPaddedClosedFormScaleValue
65    scale_nonneg := brunTitchmarshPaddedClosedFormScaleValue_nonneg

```

```

66
67 /--
68 The exact TS22 ceiling-budget scale is dominated by the padded closed-form
69 scale.
70 -/
71 theorem localWindowBudgetScale_le_paddedClosedFormScale
72   (BT : TS22.Goldbach.BrunTitchmarshNatIntervalBound) :
73   TS22.Goldbach.BrunTitchmarshScaleBridge
74     (TS22.Goldbach.localWindowBudgetOfNatIntervalBound BT)
75     brunTitchmarshPaddedClosedFormScale := by
76   refine {?_}
77   intro x Q
78   simp only
79     [TS22.Goldbach.localWindowBudgetScale,
80      TS22.Goldbach.localWindowBudgetOfNatIntervalBound,
81      TS21.Goldbach.localCountEnergyScale,
82      brunTitchmarshPaddedClosedFormScale,
83      brunTitchmarshPaddedClosedFormScaleValue]
84   have hceil :
85     (TS22.Goldbach.brunTitchmarshCeilBudget x Q : Real) <=
86       brunTitchmarshCeilKernel x Q + 1 :=
87     brunTitchmarshCeilBudget_le_kernel_add_one x Q
88   have hceil_nonneg :
89     0 <= (TS22.Goldbach.brunTitchmarshCeilBudget x Q : Real) := by
90     exact_mod_cast Nat.zero_le (TS22.Goldbach.brunTitchmarshCeilBudget x Q)
91   have hkernel_nonneg :
92     0 <= brunTitchmarshCeilKernel x Q + 1 := by
93     have h := brunTitchmarshCeilKernel_nonneg x Q
94     linarith
95   have hsquare :
96     (TS22.Goldbach.brunTitchmarshCeilBudget x Q : Real) ^ 2 <=
97       (brunTitchmarshCeilKernel x Q + 1) ^ 2 := by
98     nlinarith
99   have hmul :
100     ((x + 1 : Nat) : Real) *
101       (TS22.Goldbach.brunTitchmarshCeilBudget x Q : Real) ^ 2 <=
102       ((x + 1 : Nat) : Real) *
103         (brunTitchmarshCeilKernel x Q + 1) ^ 2 :=
104     mul_le_mul_of_nonneg_left
105       hsquare
106       (by exact_mod_cast Nat.zero_le (x + 1))
107   simpa [Nat.cast_mul, Nat.cast_pow] using hmul
108
109 /--
110 An interval-count Brun-Titchmarsh theorem yields the scaled pair-count target
111 at the padded closed-form scale.
112 -/
113 theorem Problem_E1Scale_from_natIntervalBound_paddedClosedForm
114   (BT : TS22.Goldbach.BrunTitchmarshNatIntervalBound) :
115   TS22.Goldbach.Problem_E1Scale
116     brunTitchmarshPaddedClosedFormScale
117     1 :=
118   TS22.Goldbach.Problem_E1Scale_from_localWindowBudget_bridge
119     (TS22.Goldbach.localWindowBudgetOfNatIntervalBound BT)
120     brunTitchmarshPaddedClosedFormScale
121     (localWindowBudgetScale_le_paddedClosedFormScale BT)
122
123 end Goldbach
124 end TS24

```

C.3 TS25/PaddedScaleOTSAFeasibility.lean

```

1 import Mathlib.Tactic
2 import TS.Goldbach.Strong.TS23.OTSAScalePropagation
3 import TS.Goldbach.Strong.TS24.ClosedFormScaleBridge
4
5 namespace TS25
6 namespace Goldbach
7

```

```

8  /--
9  Full local infrastructure for the padded Brun-Titchmarsh scale.
10
11  This is not an unconditional proof object. The Brun-Titchmarsh interval theorem,
12  the scale-to-OTSA transfer, and the scaled OTSA admissibility certificate remain
13  explicit local obligations.
14  -/
15  structure PaddedScaleAnalyticInfrastructure where
16    BT : TS22.Goldbach.BrunTitchmarshNatIntervalBound
17    scaleControl :
18      TS23.Goldbach.ScaleToOTSAControl
19      TS24.Goldbach.brunTitchmarshPaddedClosedFormScale
20    constants : TS23.Goldbach.ScaledOTSAConstants
21    Cscale_matches : constants.Cscale = scaleControl.Cscale
22    admissible : TS23.Goldbach.ScaledOTSAAdmissible constants
23
24  /--
25  The TS24 padded closed-form scale gives the scaled E1 estimate, assuming the
26  local interval Brun-Titchmarsh theorem.
27  -/
28  theorem Problem_E1Scale_from_padded_infrastructure
29    (H : PaddedScaleAnalyticInfrastructure) :
30    TS22.Goldbach.Problem_E1Scale
31      TS24.Goldbach.brunTitchmarshPaddedClosedFormScale
32      1 :=
33    TS24.Goldbach.Problem_E1Scale_from_natIntervalBound_paddedClosedForm H.BT
34
35  /-- Expose the scaled OTSA admissibility certificate for the padded scale. -/
36  theorem OTSA_viability_from_padded_scale
37    (H : PaddedScaleAnalyticInfrastructure) :
38    TS23.Goldbach.ScaledOTSAAdmissible H.constants := by
39    exact H.admissible
40
41  /--
42  Padded-scale OTSA residual discharge.
43
44  Once the padded scale is controlled and a local OTSA coupling estimate is
45  available for the scaled TS23 controls, the TS19 residual target follows from
46  the single admissibility certificate packaged in `H`.
47  -/
48  theorem OTSA_residual_from_padded_scale
49    (H : PaddedScaleAnalyticInfrastructure)
50    (R : TS19.OTSA.OTSAResidualFunctional)
51    (Hcoup :
52      TS19.OTSA.OTSAcouplingHypothesis
53      R
54      (TS23.Goldbach.scaledKernelSpectralControl H.constants)
55      (TS23.Goldbach.scaledTraceContributionControl H.constants)
56      (TS23.Goldbach.scaledMellinTailDecay H.constants)) :
57    TS19.OTSA.OTSAResidualBound R :=
58    TS23.Goldbach.OTSA_residual_from_scaled_constants
59      (Problem_E1Scale_from_padded_infrastructure H)
60      H.scaleControl
61      H.constants
62      H.Cscale_matches
63      R
64      Hcoup
65      H.admissible
66
67  end Goldbach
68  end TS25

```

C.4 TS26/OTSANumericalFeasibility.lean

```

1  import Mathlib.Tactic
2  import TS.Goldbach.Strong.TS25.PaddedScaleOTSAFeasibility
3
4  namespace TS26
5  namespace Goldbach

```

```

6
7 /--
8 A rational certificate for the scaled OTSA admissibility inequality.
9
10 All constants are rational upper bounds. This avoids floating-point arithmetic
11 and keeps the certificate auditable by exact rational normalization.
12 -/
13 structure OTSARationalCertificate where
14   Ck : Rat
15   Ct : Rat
16   Cm : Rat
17   Cscale : Rat
18   Ck_pos : 0 < Ck
19   Ct_pos : 0 < Ct
20   Cm_pos : 0 < Cm
21   Cscale_pos : 0 < Cscale
22   admissible_rat :
23     Cscale * (Ck * Ct + Cm) <= 26
24
25 /-- Convert a rational certificate into the real constants used by TS23. -/
26 noncomputable def scaledConstantsOfRat
27   (C : OTSARationalCertificate) :
28     TS23.Goldbach.ScaledOTSAConstants where
29     Ck := (C.Ck : Real)
30     Ct := (C.Ct : Real)
31     Cm := (C.Cm : Real)
32     Cscale := (C.Cscale : Real)
33     Ck_pos := by exact_mod_cast C.Ck_pos
34     Ct_pos := by exact_mod_cast C.Ct_pos
35     Cm_pos := by exact_mod_cast C.Cm_pos
36     Cscale_pos := by exact_mod_cast C.Cscale_pos
37
38 /-- The real scaled constant is the cast of the rational scaled constant. -/
39 theorem scaledCoupledConstant_of_rat
40   (C : OTSARationalCertificate) :
41     TS23.Goldbach.scaledCoupledConstant (scaledConstantsOfRat C) =
42       ((C.Cscale * (C.Ck * C.Ct + C.Cm) : Rat) : Real) := by
43     unfold TS23.Goldbach.scaledCoupledConstant
44     unfold scaledConstantsOfRat
45     norm_num
46
47 /-- A rational certificate proves the real TS23 admissibility condition. -/
48 theorem scaledOTSAAdmissible_of_rat
49   (C : OTSARationalCertificate) :
50     TS23.Goldbach.ScaledOTSAAdmissible (scaledConstantsOfRat C) := by
51     unfold TS23.Goldbach.ScaledOTSAAdmissible
52     rw [scaledCoupledConstant_of_rat C]
53     exact_mod_cast C.admissible_rat
54
55 end Goldbach
56 end TS26

```

C.5 TS27/OTSAConstantRegister.lean

```

1 import Mathlib.Tactic
2 import TS.Goldbach.Strong.TS26.OTSANumericalFeasibility
3
4 namespace TS27
5 namespace Goldbach
6
7 /--
8 A labelled register entry for rational OTSA constants.
9
10 The label and source note are documentation fields. The mathematical content is
11 the exact rational certificate carried by `cert`.
12 -/
13 structure OTSAConstantRegister where
14   label : String
15   status : String

```



```

16   sourceNote : String
17   cert : TS26.Goldbach.OTSARationalCertificate
18
19   /--
20   Smoke-test spectral-kernel constant.
21
22   This is deliberately not a final certified OTSA value. It records the narrative
23   candidate `Ck = 0.06 = 3/50` for testing the TS26 rational pipeline.
24   -/
25   def Ck_smoke_test : Rat :=
26     3 / 50
27
28   /-- Smoke-test trace constant. Not a final certified OTSA value. -/
29   def Ct_smoke_test : Rat :=
30     1
31
32   /-- Smoke-test Mellin-tail constant. Not a final certified OTSA value. -/
33   def Cm_smoke_test : Rat :=
34     1
35
36   /-- Smoke-test scale-transfer constant. Not a final certified OTSA value. -/
37   def Cscale_smoke_test : Rat :=
38     1
39
40   /--
41   A non-final rational certificate used only to verify the TS26 plumbing.
42
43   The admissibility check is exact:
44
45   1 * ((3/50) * 1 + 1) = 53/50 <= 26.
46   -/
47   def OTSACert_smoke_test : TS26.Goldbach.OTSARationalCertificate where
48     Ck := Ck_smoke_test
49     Ct := Ct_smoke_test
50     Cm := Cm_smoke_test
51     Cscale := Cscale_smoke_test
52     Ck_pos := by norm_num [Ck_smoke_test]
53     Ct_pos := by norm_num [Ct_smoke_test]
54     Cm_pos := by norm_num [Cm_smoke_test]
55     Cscale_pos := by norm_num [Cscale_smoke_test]
56     admissible_rat := by
57       norm_num [Ck_smoke_test, Ct_smoke_test, Cm_smoke_test, Cscale_smoke_test]
58
59   /-- Registry entry for the non-final smoke-test constants. -/
60   def OTSAResister_smoke_test : OTSAConstantRegister where
61     label := "OTSA smoke-test constants"
62     status := "smoke_test_not_final"
63     sourceNote :=
64       "Pipeline test only: Ck=3/50 follows the narrative C0 ≈ 0.058; Ct, Cm, and Cscale are placeholders."
65     cert := OTSACert_smoke_test
66
67   /-- The smoke-test register yields a TS23 admissibility proof through TS26. -/
68   theorem smoke_test_scaledOTSAAAdmissible :
69     TS23.Goldbach.ScaledOTSAAAdmissible
70     (TS26.Goldbach.scaledConstantsOfRat OTSAResister_smoke_test.cert) :=
71     TS26.Goldbach.scaledOTSAAAdmissible_of_rat OTSAResister_smoke_test.cert
72
73   end Goldbach
74   end TS27

```

C.6 TS28/OTSAConstantsCandidate.lean

```

1   import Mathlib.Tactic
2   import TS.Goldbach.Strong.TS27.OTSAConstantRegister
3
4   namespace TS28
5   namespace Goldbach
6
7   /--

```

```

8   Status tag for rational OTSA constant registers.
9
10  This separates plumbing tests, analytic candidates, and fully certified
11  constant packages.
12  -/
13  inductive ConstantStatus where
14    | smoke_test
15    | analytic_candidate
16    | certified
17    deriving DecidableEq, Repr
18
19  /--
20  Labelled rational OTSA constant register with a typed status.
21
22  The certificate field is still checked by TS26. The status records how much
23  analytic provenance is attached to the rational bounds.
24  -/
25  structure LabelledOTSAConstantRegister where
26    label : String
27    status : ConstantStatus
28    sourceNote : String
29    cert : TS26.Goldbach.OTSARationalCertificate
30
31  /--
32  Candidate v0 spectral-kernel constant.
33
34  This keeps the narrative upper bound `C0 ~ 0.058` padded to `3/50`.
35  -/
36  def Ck_candidate_v0 : Rat :=
37    3 / 50
38
39  /--
40  Candidate v0 trace constant.
41
42  This remains a placeholder until a trace-control source is registered.
43  -/
44  def Ct_candidate_v0 : Rat :=
45    1
46
47  /--
48  Candidate v0 Mellin-tail constant.
49
50  This remains a placeholder until a Mellin-tail source is registered.
51  -/
52  def Cm_candidate_v0 : Rat :=
53    1
54
55  /--
56  Candidate v0 scale-transfer constant.
57
58  This remains a placeholder until a scale-to-OTSA source is registered.
59  -/
60  def Cscale_candidate_v0 : Rat :=
61    1
62
63  /--
64  Candidate v0 OTSA rational certificate.
65
66  This is not a final certificate: the rational inequality is certified, but the
67  analytic provenance of `Ct`, `Cm`, and `Cscale` is still to be supplied.
68  -/
69  def OTSACert_candidate_v0 : TS26.Goldbach.OTSARationalCertificate where
70    Ck := Ck_candidate_v0
71    Ct := Ct_candidate_v0
72    Cm := Cm_candidate_v0
73    Cscale := Cscale_candidate_v0
74    Ck_pos := by norm_num [Ck_candidate_v0]
75    Ct_pos := by norm_num [Ct_candidate_v0]
76    Cm_pos := by norm_num [Cm_candidate_v0]
77    Cscale_pos := by norm_num [Cscale_candidate_v0]
78    admissible_rat := by
79      norm_num [Ck_candidate_v0, Ct_candidate_v0, Cm_candidate_v0, Cscale_candidate_v0]
80

```

```

81 /-- Labelled candidate-v0 register. -/
82 def OTSAResister_candidate_v0 : LabelledOTSACandidateRegister where
83   label := "OTSA candidate constants v0"
84   status := ConstantStatus.analytic_candidate
85   sourceNote :=
86     "Candidate only: Ck=3/50 follows the narrative C0 ≈ 0.058; Ct, Cm, and Cscale must be replaced by sourced
87     ⇔ rational majorants before certification."
88   cert := OTSACert_candidate_v0
89
90 /-- The candidate-v0 certificate proves the TS23 admissibility inequality. -/
91 theorem candidate_v0_scaledOTSAAAdmissible :
92   TS23.Goldbach.ScaledOTSAAAdmissible
93   (TS26.Goldbach.scaledConstantsOfRat OTSACert_candidate_v0) :=
94   TS26.Goldbach.scaledOTSAAAdmissible_of_rat OTSACert_candidate_v0
95
96 /-- The candidate-v0 register carries the expected analytic-candidate status. -/
97 theorem candidate_v0_status :
98   OTSAResister_candidate_v0.status = ConstantStatus.analytic_candidate :=
99   rfl
100
101 /-- The candidate-v0 register also feeds TS23 through TS26. -/
102 theorem candidate_v0_register_scaledOTSAAAdmissible :
103   TS23.Goldbach.ScaledOTSAAAdmissible
104   (TS26.Goldbach.scaledConstantsOfRat OTSAResister_candidate_v0.cert) :=
105   TS26.Goldbach.scaledOTSAAAdmissible_of_rat OTSAResister_candidate_v0.cert
106
107 end Goldbach
108 end TS28

```

C.7 TS29/OTSACandidateProvenance.lean

```

1  import Mathlib.Tactic
2  import TS.Goldbach.Strong.TS28.OTSACandidate
3
4  namespace TS29
5  namespace Goldbach
6
7  /--
8  Source status for an OTSA rational upper bound.
9
10 This metadata lives in Lean so that placeholders, narrative bounds, numerical
11 experiments, analytic derivations, and Lean-certified constants remain
12 distinguishable.
13 -/
14 inductive ConstantProvenance where
15   | placeholder
16   | narrative_source
17   | numerical_experiment
18   | analytic_derivation
19   | lean_certified
20   deriving DecidableEq, Repr
21
22 /-- A sourced rational upper bound for one OTSA constant. -/
23 structure SourcedRatBound where
24   value : Rat
25   provenance : ConstantProvenance
26   label : String
27
28 /-- Provenance register for the four OTSA constants. -/
29 structure OTSACandidateProvenanceRegister where
30   Ck : SourcedRatBound
31   Ct : SourcedRatBound
32   Cm : SourcedRatBound
33   Cscale : SourcedRatBound
34   packageStatus : TS28.Goldbach.ConstantStatus
35
36 /--
37 Candidate-v0 provenance register.
38

```

```

39 Only `Ck` currently has a narrative source. The other constants remain
40 placeholders until trace, Mellin-tail, and scale-transfer majorants are sourced.
41 -/
42 def OTSAProvenance_candidate_v0 : OTSAConstantProvenanceRegister where
43   Ck :=
44     value := 3 / 50
45     provenance := ConstantProvenance.narrative_source
46     label := "Ck candidate: spectral/KLMN narrative bound C0 ≈ 0.058, rounded to 3/50"
47   Ct :=
48     value := 1
49     provenance := ConstantProvenance.placeholder
50     label := "Ct placeholder: trace contribution bound not yet sourced"
51   Cm :=
52     value := 1
53     provenance := ConstantProvenance.placeholder
54     label := "Cm placeholder: Mellin-tail bound not yet sourced"
55   Cscale :=
56     value := 1
57     provenance := ConstantProvenance.placeholder
58     label := "Cscale placeholder: padded-scale transfer cost not yet sourced"
59   packageStatus := TS28.Goldbach.ConstantStatus.analytic_candidate
60
61 /-- The candidate-v0 provenance package is not certified. -/
62 theorem candidate_v0_not_certified :
63   Not (OTSAProvenance_candidate_v0.packageStatus =
64     TS28.Goldbach.ConstantStatus.certified) := by
65   decide
66
67 /-- Candidate v0 records `Ck` as a narrative-source bound. -/
68 theorem candidate_v0_Ck_provenance :
69   OTSAProvenance_candidate_v0.Ck.provenance =
70     ConstantProvenance.narrative_source :=
71   rfl
72
73 /-- Candidate v0 records `Ct` as a placeholder. -/
74 theorem candidate_v0_Ct_placeholder :
75   OTSAProvenance_candidate_v0.Ct.provenance =
76     ConstantProvenance.placeholder :=
77   rfl
78
79 /-- Candidate v0 records `Cm` as a placeholder. -/
80 theorem candidate_v0_Cm_placeholder :
81   OTSAProvenance_candidate_v0.Cm.provenance =
82     ConstantProvenance.placeholder :=
83   rfl
84
85 /-- Candidate v0 records `Cscale` as a placeholder. -/
86 theorem candidate_v0_Cscale_placeholder :
87   OTSAProvenance_candidate_v0.Cscale.provenance =
88     ConstantProvenance.placeholder :=
89   rfl
90
91 end Goldbach
92 end TS29

```